Serial Brun — starting a bench soak from the front panel

Ian Ameline · version 1.76 · MIT licence

`Serial_brun.tspa` puts the bench engine behind a settings screen on the instrument, so a lap can be started with nobody connected. It is the same `brun.soak()` the host drives — a second door into one engine, not a second engine. Everything `tools/run_bench.py` did still works, and a host-driven load leaves this screen available for the next lap.

It is bench scaffolding and is not the Serial Decode app. It carries the whole app inside it, because the engine measures through `sdec.acquire()`, but it installs separately and shows its own screen. Do not install it on an instrument you want as a serial analyser.

Starting up: the key, then the plan

Three things happen before the form appears, and each has a screen of its own because each can fail in a way the operator has to act on.

The USB key

The app keeps its plan *and* its record on `/usb1`, so with no key there is nothing it can usefully do. It checks at startup and, if the key is absent, says so and offers two answers: **Retry** and **Exit**. Retry re-checks and reports the attempt even when nothing has changed — `Still no key at /usb1 — checked 3 time(s)` — because a button that silently does nothing is indistinguishable from a dead one, and the next move after a dead button is the power switch.

The plan — and it draws its own now

Until version 1.76 this app could not draw a plan, and this document argued it should never learn: re-deriving the draw on the instrument would be a second implementation of the one thing every lap is compared against, since the amplitude, the offset and the wait are all keyed off a vector's index, so a plan drawn twice is two different experiments wearing one name.

That objection was right, and it is answered by construction rather than by care. The app draws its own plan now, and nothing about the draw is a second copy:

- **the generator is not a copy.** `bench/plan_rand.tsp` is the MT19937 that `tools/test_soakrand.py` holds to CPython's own sequence, *moved* out of `tools/` rather than duplicated — so that proof now covers the code that ships.
- **the data is not a copy.** `bench/plan_data.tsp` is generated by `tools/gen_plan_data.py` *from* `soakplan.py`, so the vector list — whose order is what the keying depends on — and every constant are the host's own values. A vector added to `vector_names.py` without regenerating fails the test suite rather than a lap.
- **the ladder is not a copy.** `sdec.rates`, `sdec.stdbaud` and `sdec.flatfloor` are read live on the instrument and read out of `tsp/serial_core.tsp` on the host. Neither side holds a snapshot.
- **and the draw is checked, not reasoned about.** `tools/test_plan_gen.lua` runs the instrument's draw under both Luas and compares the result against `soakplan.py --emit-csv` **byte for byte**, including the per-lap random subset, repeats, vector subsets and an empty skip. It is the `unit-plangen` stage of `release_sweep.py`.

So the remaining risk is named rather than hidden: this is an independent implementation whose equivalence rests on a test. The two draws are the same experiment exactly as long as that stage runs.

It draws only when what is on the key cannot be used, and `--spec` pushes are therefore safe: a plan the operator chose is left alone. Drawing happens when the key has no plan, no row count, no column header, or **fewer rows than it declares** — a plan truncated by a key pulled mid-push opens, parses and streams, and then ends early, so a lap would silently run a prefix and the record would read as a short run rather than a broken file.

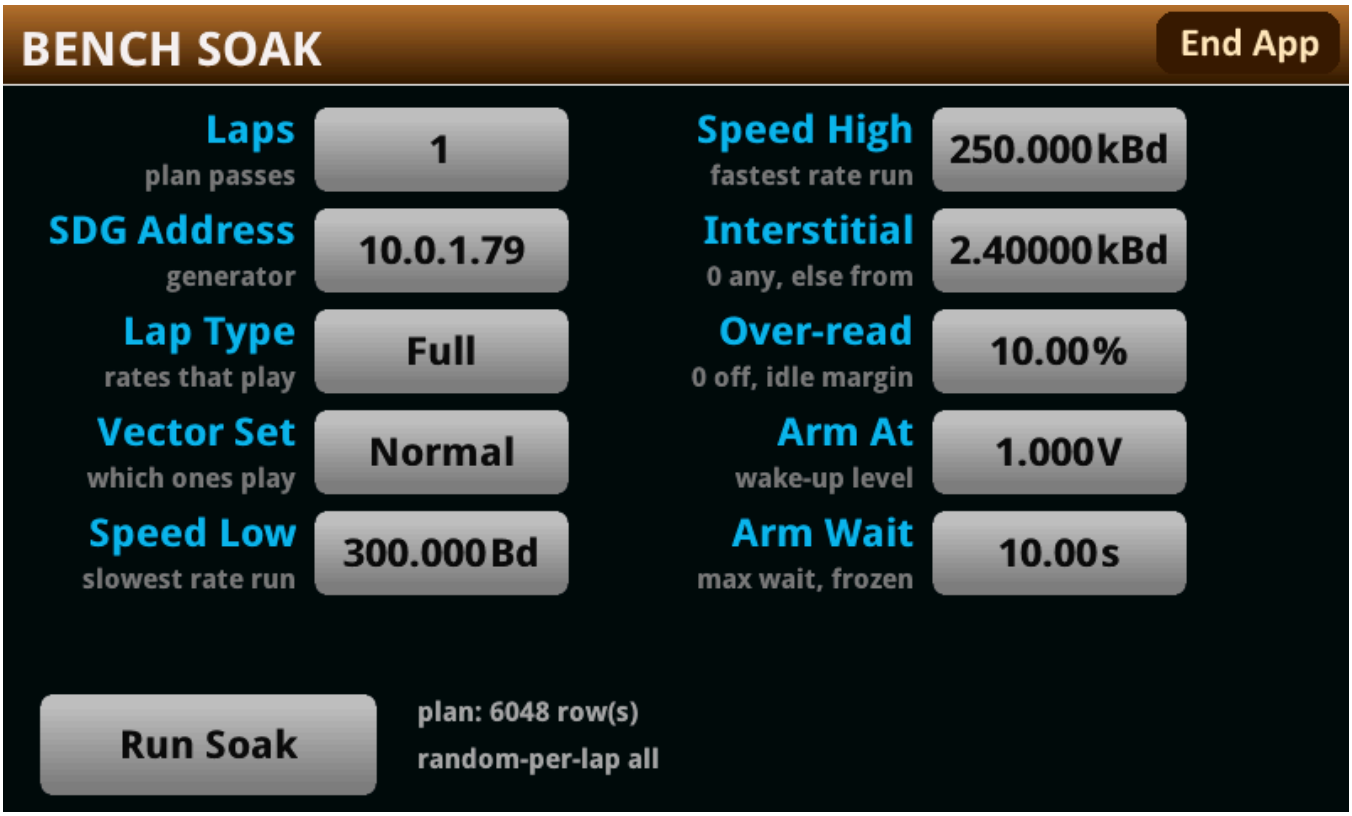
It is not fast, and it says so while it works. The draw keys a fresh MT19937 stream per cell, and there are thousands of cells: 5.2 s for a 1512-row plan under the host Lua, and the instrument is slower than the host. So the screen names what it is doing, counts the rows, and carries a spinner at a 4 Hz ceiling, driven per row rather than per block so its rate does not depend on how long a block takes. A form that came up blank for minutes would be indistinguishable from a wedged app.

The host path is unchanged and still the way to put a *chosen* plan on the key:

```
python3 tools/run_bench.py --push-plan                          # the full draw
python3 tools/run_bench.py --push-plan --spec v98d:std,v99a:std # a targeted subset
```

A `--spec` push **overwrites** `PLAN.CSV`, so the row count on the screen is the only thing that says which plan is actually on the key.

The screen



There is no Cancel button. `End App` in the title bar is the way out, and it is wired to the same teardown Cancel called — which is a complete exit here rather than a screen change, because the bench package's entry point calls `brun.panel()` and never `sdec.start()`, so the form is the only thing this app has built.

The plan runs in two short lines beside **Run Soak**, which is dead space in a single-button row and is what made room for ten fields; it used to take a field slot of its own. It is read off the key when the screen is built, so it describes what is actually there rather than what was last pushed from this host. The count is the plan's own `# rows=` header — data rows, which is six fewer than the line count `run_bench.py --push-plan` reports, that figure including the header and comment lines.

field	what it does
Laps	passes over the plan, 1 to 210. There is deliberately no "forever": an unbounded run started at the panel has nobody watching a host log that would say so.
SDG Address	the generator, as a pick list. There is no string-entry object in this firmware's set, and a list cannot produce the typo that costs a lap to discover. The live value leads it.
Lap Type	narrows the plan — see below.
Speed Low / **Speed High**	the rate window in Bd, inclusive at both ends. Defaults span the ladder: 300 is the slowest rate drawn, 250 000 the decoder's own ceiling.
Interstitial	a floor under the *drawn* rates only, `0` for none. Standard rates still play below it. At the bottom of the ladder a non-standard rate costs the most seconds and tests the least.
Vector Set	which vectors play — see below. Defaults to **Normal**.
Over-read	the idle margin on every recording, as a percentage, `0` for off. The app's own `sdec.strm_overread`, which until now could only be set from the host with `run_bench.py --over-read`.
Arm At	the comparator level that wakes a capture on a line which has not started talking. `sdec.armlevel`, 0.33 to 6 V.
Arm Wait	how long an armed capture waits, and the quiet-line exit once the line is up. `sdec.armwait`, 2 to 120 s. **On a soak this is seconds per cell**, so it is the field most able to turn a lap into an overnight run.

The last four are the app's own capture settings rather than the bench's, and they are applied the way the Options form applies them — the arm pair unclamped, because `arm_level_v()` and `arm_wait_s()` clamp at use so the form keeps showing what was typed, and Over-read clamped to `overread_max` because that ceiling is a field argument and a form built before it moved would still offer the old one.

Lap Type, and Vector Set

Two axes, and they compose. Until 2026-10-21 one field carried both: `Lap Type` read Full / Normal / Adversarial, where *Normal* filtered rates and *Adversarial* filtered vectors. So "adversarial vectors at standard rates only" could not be asked for at all. `Lap Type` is now the rate axis alone and `Vector Set` is the vector axis.

Lap Type	what plays
Full	every rate the plan holds, which is what the host-driven path runs
Standard only	standard rates only — no drawn interstitial or edge rate

Vector Set	what plays
Normal	the default. No gap vectors and nothing `barb.loud` excuses — the lap you want when the question is whether the decoder still works

Vector Set	what plays
All	every vector the plan holds
Start gap	a gap *before the first byte*: the `GapLead` pair, which is the armed-start case
All gaps	every gap vector — lead, mid, both, and the long ones
Adversarial	only the vectors known to produce errors, so a lap spends its time where the answers are

Three of those are **derived, not listed**. `Adversarial` is `barb.expect`'s `loud` entries — the vectors the plan drives into conditions where refusing is the documented right answer. `Start gap` and `All gaps` key on the generator's own waveform **name** through `barb.of`, every gap vector carrying `Gap` in its, so a gap vector added later is covered without editing a second list.

A vector the name table does not know is run, not skipped. `barb.of` answers nil only when `arb_names.tsp` is stale, and a stale table should fail loudly at the `ARWV` that cannot find the waveform rather than quietly shrink the lap — which is the one outcome nobody would notice.

Running one

Press **Run Soak**. The settings screen is torn down and the engine's own status screen takes over, which is the same screen a host-driven lap shows: lap and cell, the waveform playing, the measured rate, the error line and a spinner that keeps moving through the parts of a capture that take seconds.

Two refusals can appear on the plan line instead, and both are worth reading rather than retrying:

`REFUSED: no plan on the key` — nothing to run. Push one.

`REFUSED: a smoke has run on this power cycle` — a soak started after a smoke fails *every* cell with `acq: line is idle (no transitions)`, and only a power cycle clears it. This has happened three times on this bench; the last was 705 cells of 705. The host guard that catches it lives in `run_bench.py`, so the panel carries its own.

Stopping is the **`Abort`** check box on the status screen. Tick it and the run ends after the current waveform and closes the record, which is what commits it on a FAT key. Cutting the power costs at most the last row.

The front-panel **TRIGGER** key also stops a run, but **only one started from the host.** That is not a preference, it is what the firmware does, and it is worth stating plainly because the panel spent a while promising a control that could not work:

the run was started by	the TRIGGER key	a button's handler	the `Abort` box
the host, or any top-level call	**latches** — seen at iterations 23, 59 and 11 of three polled loops, with a trigger model loaded and with none at all	never dispatched	read every cell
a press on the glass, so a display handler	**never**	never dispatched	**read every cell**

Measured on 1.7.17a, 2026-10-22, three arms each. While Lua is executing the firmware *queues* front-panel input — the glass clicks and nothing happens — and it routes the TRIGGER key to the trigger subsystem only when the running code is not inside a display handler. A panel-started soak always is, which is why three presses against one did nothing, and why no `OBJ_BUTTON` can abort a run either. An

edit object is different because the **firmware owns its value** and keeps updating it from the glass regardless: an `OBJ_EDIT_OPTION` was seen changing at iteration 38 of 160 with no event posted, and the check box used here is the same mechanism.

`display.waitevent` is not an alternative. It wedges this firmware at a timeout of `0.001`, not only at the documented `0`, and the only way out is the power.

Two things make the abort prompt rather than merely possible. `brun.holdslice` is 2 s, because that interval is also how often a parked run re-reads its stop controls — at 10 s an abort took up to ten seconds to be noticed. And `bsdg.ask` polls an optional `bsdg.giveup` hook about twice a second inside its own 50 ms loop, so a stop does not queue behind a wedged generator's 5 s timeout, several of which fall in every cell. `brun.soak` points that hook at `brun.ab_hit` and deliberately **not** at `brun.stopasked`: reading the box is idempotent, where polling the TRIGGER key's blender latch clears it, so the other choice would silently eat a press between cells.

The status screen

`Run Soak` tears the form down and `brun.ui_build()` takes over. It is the same screen a host-driven lap shows — the title changes with the run's tag, so a smoke reads `SD SMOKE` and a soak `SD SOAK`. Nine lines, and every one of them answers a different question. The bottom row is shared: the **`Abort`** check box on the left, with the error count beside it rather than on a row of its own, which is what buys the log its fifth line back.

The title is **`SD <TAG>`**, and the tag has all 31 characters to itself. It used to carry a stop instruction — first `TRIGGER TO STOP`, which was true only for a host-started run, then `ABORT ON SCREEN`, which was true but redundant once the `Abort` box carried its own label where the finger is. The tag is what a title is for here: it is the one thing that tells a smoke from a soak from a resumed run at a glance.

line	reads	what it is for
headline	`SOAK HEALTH 97% RUNNING 5420k`	the answer from across the room: health, the run's state in a word, and the Lua heap. Red if it gave up or the heap is past `heapbadk`, green while healthy, amber when either is borderline, cyan before there is a figure at all.
spinner	`|` top right	one character, redrawn six times a cell, driven from inside the app's long loops. If it stops moving the instrument is wedged, not merely slow — it keeps turning through `sdec.acquire()`, which is 58 % of a cell and where a hang is most likely.
lap / cell	`L1 C86 r86 Ran: 4m Left: 2m`	position in the lap, then the lap's own elapsed and remaining. `r86` is the plan's **own** cell label and appears only when it differs from the position — the record writes rows as `L<iter>C<label>`, so that is the number to search for.
totals	`Total: 12m of 1.4h – 86/1512 c 6%`	elapsed of the **projected** total, then cells done out of the whole run. Not a second countdown: "left 853.8h" says nothing about how long the thing was ever going to be.
stimulus	`v98d 9600 Bd 8N1 SER_LoremGap100ms_8N1_x10`	the vector, the commanded rate, the expected format and the generator's **own** waveform name. The full name rather than the short form, because that is what `ARWV NAME` was sent and the one thing that can be checked against the generator.

line	reads	what it is for
refusals	`Refusals: Allowed 12 Unexpected 0 of 86 cells`	cells that refused to decode. `Allowed` is the vectors the plan drives into conditions where refusing is correct; `Unexpected` is the count that matters.
vector	`Vec: v98d 4 of 20 bad  I3 L0 S1`	this vector's own failure count beside the run's, because the two answer different questions — whether the soak is healthy, and whether **one** waveform is the reason it is not. `I`, `L`, `S` are idle, levels and swing; initials because spelled out at six-figure totals the line is 70 characters against the 58 the glass holds.
log	five scrolling lines	stamped with the lap and cell they happened at. This is what says *when* something started, which is the row that pays for itself on a run nobody is standing at.
errors	`Errors: 22  Bad: 640/17160` then `nodec 10  flag 7  wrong 5/132`	see below. **Two rows**, stacked at x=380 to the right of the `Abort` box.
abort	a tick box labelled `Abort`	the only stop control a panel-started run can see. Bottom left, hard against the bottom edge.

Resume, or Repeat

The result screen carries one button beside `Back`, and which one depends on how the run ended:

the run	the button	what it does
was cut short — aborted, or parked	**`Resume`**	carries on from the cell after the last one the record holds, tagged `resumed`
finished what it was asked for	**`Repeat`**	another go from the top with the same settings, tagged `soak`

Never both. They share a position and exactly one is visible, because resuming a completed run would replay the whole plan under a header saying `resumed`, and repeating one that was cut short would throw away the cells it did measure.

Two buttons rather than one whose label is rewritten. `display.settext` on a *screen* sets its title and that is measured, but the button case is an assumption on this firmware — and a button reading `Resume` while it repeated the run is worse than no button. Hiding with `STATE_INVISIBLE` is how `tsp/serial_ui.tsp` shows and hides its own buttons, so it is already proven here.

Every option holds through. Both paths go back through `brun.panel_run`, which reads the form again, so anything changed before the run stays changed for the resumption or the repeat — the filters, the rate window, the three capture settings, the generator address and the lap count. That is asserted offline with deliberately non-default values, because options equal to their defaults would carry through whether the code preserved them or not.

`Resume` carries on from the cell after the last one the record holds. That is worth days of stimulus after an abort, and until it existed the only way to do it was the host's `--resume` — the dependency this app exists to remove.

Which button appears is decided by `brun.resumable()` — `brun.stopuser`, set by `brun.stopasked` when either stop control fires, or `brun.stopbad` where the run parked. Deliberately **not** read off `brun.stopwhy`, which is non-nil for a natural ending too (`1 iteration(s) complete`), so matching text in

it would be a rule living in a string.

Hidden rather than disabled: `display.STATE_DISABLE` applies to `display.OBJ_EDIT_*` only on this firmware, so a "disabled" button is simply still live.

Two things about it are load-bearing:

- **It goes back through `brun.panel_run`**, so the form is read again and every filter, capture setting and generator address is applied exactly as a fresh run applies them. A resume that built its own configuration would be a second place for the plan's cell set to be decided, and a resumed leg measuring a different set than the leg before it is worse than no resume — the record carries both under one header.
- **The leg is tagged `resumed`.** `judge_bench.split_runs` treats `resumed` and `roll` as *continuations*; any other tag starts a new run in the record's header and the judge then reads only the last one. Tagged `soak`, a resume silently halves what is judged. `brun.resume` is set on **every** start, not only a resuming one, or a later Run Soak would inherit the flag and carry on from the record while reporting a full lap count.

The engine does the rest and is already careful: `brun.resume_scan` finds the last completed cell bounded by the record's own FAT size, so it cannot read past the end, and `brun.resume_to` **refuses** a resume point the plan does not contain — that means the plan is not the one that produced the record, and starting over would replay days of stimulus while reporting a resume.

The error line is drawn as **two** rows because the width beside the box is `brun.errw` 31 characters — x=380 to the right edge, at a font advancing about 13.4 px — and the line reaches 56 at its widest. The split follows the priority the single row already had: the total and the byte count lead the first row, the per-class breakout goes on the second, and the call to action is appended to the second and is **never** what gets dropped when it will not fit. All four values come from one call to `brun.errsline()`, so the two rows and the record's own note cannot disagree about what was counted.

The geometry is four constants and the rest is derived: `brun.rowdy` 32 is the row pitch, `brun.msgy0` 216 the top of the log block, `brun.aby` 364 the bottom row, and `brun.errw` 31 the width of one slot beside the box. **`brun.aby` is fixed, not derived** — the box is against the bottom edge, so the log fits above it and does not push it down. Both bounds are asserted offline, because breaking either is silent: `display.create` returns nil for a y the firmware will not take and every later object on that screen is nil too, with nothing said. Deriving `aby` from the row count put the box 12 px below the edge; before that the log was cut to three rows because nobody worked out that five still fitted.

The error line

`Errors:` leads with the total of all three classes. Each class is written only when it is nonzero, so a clean run reads `Errors: 0` exactly and a reader's eye is not trained to skip a row of zeroes.

- **`Bad: b/t`** — payload **bytes**, wrong over compared. This is the one to read first: `wrong 5/132` says five cells are bad and nothing about whether that is a byte each or a whole payload each.
- **`nodec N`** — cells that decoded nothing on a vector that should decode.
- **`flag N`** — cells where the app itself flagged a large share of its frames. Healthy cells run 0 to 2 % flagged and the wrong ones 30 to 55 %, so this is the class that shows trouble earliest.
- **`wrong N/M`** — cells whose bytes were not the payload, over cells that could be judged at all. The denominator is load-bearing: a zero with no denominator is not a pass, it is a question nobody asked.

`Bad:` and `wrong:` are derived from the same comparison inside one gate, so they always describe the same set of cells. When the line will not fit the glass the per-class breakout is dropped and the count and the byte figure are kept — a bad run is exactly when this line is widest, so a byte figure living in the droppable part would be invisible whenever it mattered.

The panel is not the oracle. The error line is a health indicator computed on the instrument; the verdict comes from the host judge below.

The result screen

When a run ends, one screen says what it found — on both endings, because the engine has a single exit and laps-completed, `STOP.TXT`, the cell cap, a generator that gave up and **TRIGGER** all arrive there. TRIGGER is the common one on an overnight soak.

It answers the two questions an operator walking up to a finished run actually has.

Did anything get worse? The headline reads **`NO REGRESSIONS`** in green, `N REGRESSION(S)` in red, or `NO PREVIOUS RUN TO COMPARE` in amber — which is deliberately *not* green, because a first run on a fresh key has nothing to compare against and reporting that as a pass would make every new key look clean. Each finished run appends a line to `/usb1/SERDEC/EVAL.CSV`, so the comparison is against the run before it, and the file is a short history rather than a single value.

The comparison is against rates, never counts. Runs differ in length — five laps against one, or a TRIGGER press at cell 300 — so raw totals compare two different experiments. Each of the four classes is normalised by *its own* denominator, and the denominators are not interchangeable: `wrong` is cells over cells that could be **judged**, `badb` is bytes over bytes **compared**, `flag` and `unexp` are cells over cells **run**. Dividing all four by the cell count would let a regression in the judged subset hide behind the vectors that cannot be judged.

A class is called a regression only when **both** hold: the rate rose by more than 25 %, *and* the class gained at least 3 occurrences over what the previous rate predicted for this run's length. The second condition is what stops a short run reporting a regression off one extra bad cell — at 40 cells, one cell is a 2.5 % rate against a previous 0.5 %, which is a 400 % rise and is one cell. Both thresholds are fields (`beval.rel`, `beval.absmin`) and both are on the screen, because this is a panel light an operator should be able to argue with and not a significance test. It is deliberately not one: lap failure counts on this bench are **under**-dispersed against binomial by 3.09×, so a binomial test would be both too lenient and a claim this screen is in no position to make.

Which waveforms are responsible? The worst **six** vectors, with the count the run has for each, and the total number that had any error at all. The ordering is a **priority, not a sum**: `wrong` , then `bad` bytes, then `flag` , then unexpected refusals. One wrong cell therefore outranks two hundred flagged frames — wrong bytes are a decode that confidently lied, flagged frames are a decode that said it was unsure — and summing the classes would let the loudest bury the most serious. Vectors with nothing to report are not listed: six rows padded with clean vectors reads as six problems.

The body is `FONT_3` throughout, and the six rows are **built empty at launch and only ever `settext`'d**, so a run in which sixty vectors had errors costs exactly the same eleven display objects as one in which none did.

`Back` returns to the settings form, so a finished lap is not a dead end and another can be started without relaunching the app. The plan line is re-read on the way back, because a record has just been written to the key and on a long soak the key is also the thing most likely to have been swapped.

Every screen is built at launch, and nothing is deleted until the app exits

All five — the cover, the key screen, the working screen, the settings form and the result screen — are created once, at startup, behind a two-object cover. Creating a screen makes it the active one, so each switches back to the cover before its first child: without that, the form assembled itself in front of the operator thirteen objects at a time. The irreducible exposure is the cover's own single line of text, and the offline suite pins it at exactly that.

Nothing is torn down between uses. The status screen used to be deleted and rebuilt on every run and the result screen around every use; both are now built once and `settext`'d, so **a second lap in one session costs no display object at all** — which matters because `Back` makes several laps per session the normal way to work. The app exiting is the only thing that deletes, and it frees all five.

That change found a defect it was not looking for: the status screen's title is created inside a `pcall`, and a tag long enough to push the title past the firmware's 31-character limit made the *whole status screen* fail to build, silently, for the length of the run. The tag is truncated to fit now. It was then truncated to leave room for a stop instruction as well; since that phrase has gone from the title the tag gets the full 31, so it takes a much longer tag to reach the ceiling — which is why the test that guards this uses one well past it rather than one that merely nears it.

It is still not the oracle, and it says so on the screen. These are the engine's own counters, computed cell by cell with the thresholds in `bench_run.tsp`; the host judge weighs coverage floors and head damage this cannot see. A green headline means *nothing got worse by the panel's own measure*, not *passed*.

Reading the result

The record is `/usb1/SERDEC/SOAK.csv` on the key and the panel is not the oracle — the error line is a health indicator, and a lap's real verdict comes from the host judge:

```
python3 tools/run_bench.py --fetch --no-load
python3 tools/judge_bench.py --verbose out/bench/SOAK.csv
```

The error line reads `Errors: N  Bad: b/t  nodec N  flag N  wrong N/M`. `Bad:` is bytes and `wrong:` is cells, and the difference matters: `wrong 5/132` says five cells are bad, where `Bad: 251/1313` says four fifths of the bytes were fine and the damage is localised. When the line will not fit the screen the per-class breakout is dropped and the count and the byte figure are kept.

Installing

```
python3 tools/package_tspa.py --bench          # writes Serial_brun.tspa
```

Copy it to the key's root and install it from **Menu > Scripts**. It needs the same USB key the record and the plan live on.

What it does not change

The host path is untouched. `run_bench.py --start --detach` behaves exactly as before, including its own smoke-poison guard and the key pruning it does first. The panel's settings are applied to the same `brun.*` fields the host sets, and every filter defaults to nil — so a host-driven lap reads the rows it always did, whatever was last chosen on the glass.